

MrDocs: Reliable Reference Documentation for C++ Using the Clang AST

ALAN DE FREITAS

You know the most about a function the moment
you write its signature.

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	×	✓
Friendly output (user)	✓	✓	×
AI-friendly	✓	×	×
IDE integration	✓	×	×
Verifiable	✓	×	×
In sync with source	✓	×	×
Single source of truth	✓	×	×

	Generated	By hand	None
Convenient (author)	✓	✗	✓
Friendly output (user)	✓	✓	✗
AI-friendly	✓	✗	✗
IDE integration	✓	✗	✗
Verifiable	✓	✗	✗
In sync with source	✓	✗	✗
Single source of truth	✓	✗	✗

	Generated	By hand	None
Convenient (author)	✓	×	✓
Friendly output (user)	✓	✓	×
AI-friendly	✓	×	×
IDE integration	✓	×	×
Verifiable	✓	×	×
In sync with source	✓	×	×
Single source of truth	✓	×	×

	Generated	By hand	None
Convenient (author)	?	×	✓
Friendly output (user)	?	✓	×
AI-friendly	?	×	×
IDE integration	?	×	×
Verifiable	?	×	×
In sync with source	?	×	×
Single source of truth	?	×	×

Functions

```
detail::scope_token scoped_context (char const *key, char const *value)
```

```
template<class T>
```

```
std::enable_if_t< std::is_integral_v< T >, T > twice (T x)
```

Multiply by two, but only for integers.

Variables

```
constexpr clamp_fn clamp = {}
```

Constrain a value to a range.

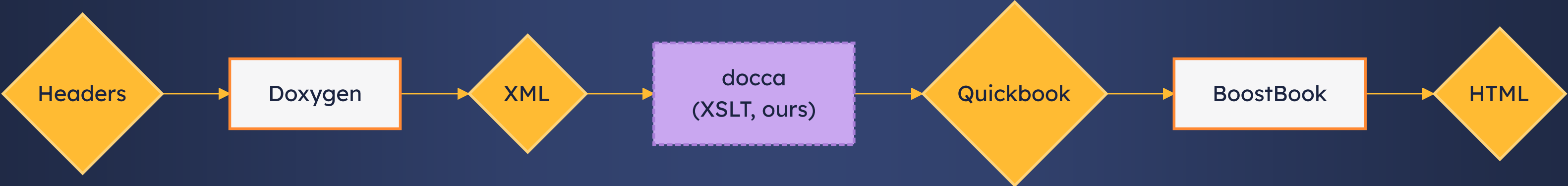
```
1 #ifdef BOOST_DOXYGEN
2 template<class T> T twice(T x);
3 #else
4 template<class T>
5 std::enable_if_t<std::is_integral_v<T>, T> twice(T x);
6 #endif
```

```
1 #ifdef BOOST_DOXYGEN
2 template<class T> T twice(T x);
3 #else
4 template<class T>
5 std::enable_if_t<std::is_integral_v<T>, T> twice(T x);
6 #endif
```

```
1 #if BOOST_BEAST_DOXYGEN
2 __implementation_defined__
3 #else
4 detail::bind_wrapper<
5     typename std::decay<Handler>::type, Args...>
6 #endif
7 bind_handler(Handler&& handler, Args&&... args);
```

```
1 class stream
2 #if ! BOOST_BEAST_DOXYGEN
3     : private stream_base
4 #endif
5 {
```

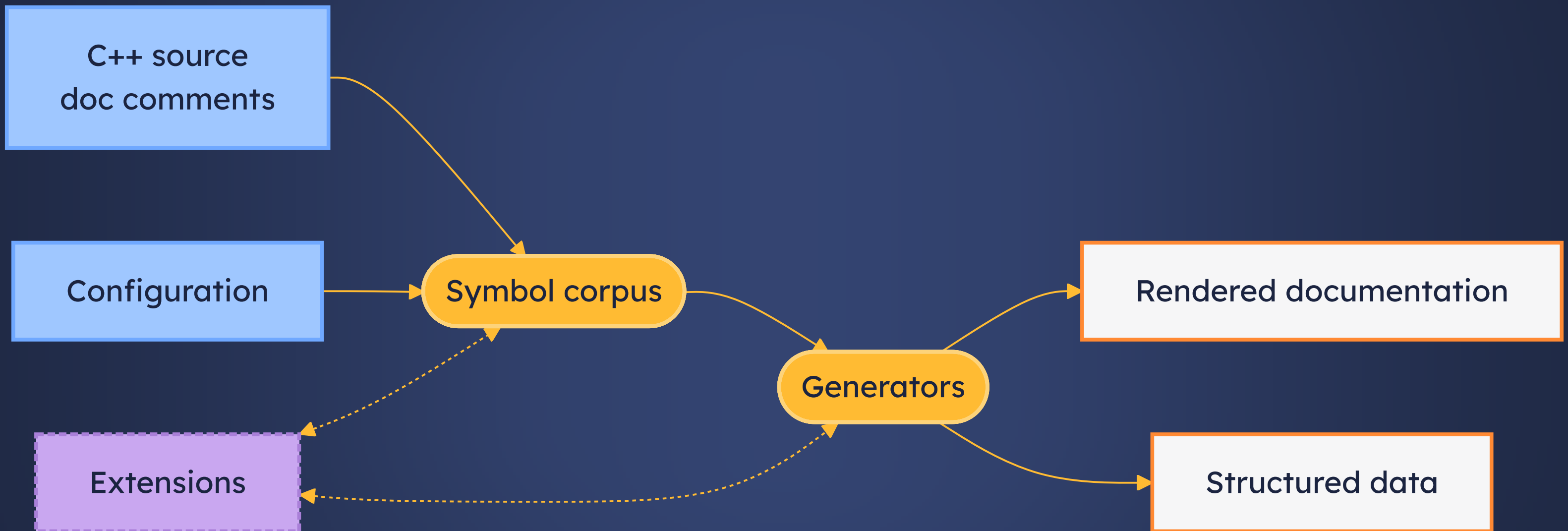
```
1 #if BOOST_BEAST_DOXYGEN
2 template<class... BufferSequence>
3 buffers_cat_view<BufferSequence...>
4 buffers_cat(BufferSequence const&... buffers)
5 #else
6 template<class B1, class... Bn>
7 buffers_cat_view<B1, Bn...>
8 buffers_cat(B1 const& b1, Bn const&... bn)
9 #endif
```



What if we just used the compiler?



MR.DOCS





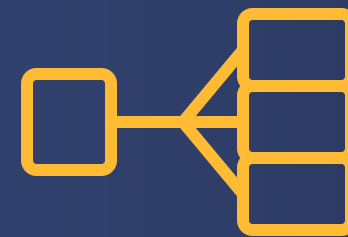
Getting Started



Commands



Configuration



Generators



Extensions



Boost.URL

Quick Look

URLs

- Parsing
- Components
- Segments
- Params

Operations

- Normalization
- String Token
- Percent Encoding
- Formatting

Customization

- Parse Rules
- Character Sets
- Compound Rules
- Ranges
- RFC 3986

Examples

- QR Code
- Finicky
- mailto URLs
- Magnet Link
- File Router
- Router
- Sanitizing URLs

Design Rationale

Reference

Help Card



boost::urls::url_view



boost::urls::url_view

A non-owning reference to a valid URL

Synopsis

Declared in `<boost/url/url_view.hpp>`

```
class url_view
    : public url_view_base
```

Description

Objects of this type represent valid URL strings constructed from a parsed, external character buffer whose storage is managed by the caller. That is, it acts like a `core::string_view` in terms of ownership. The caller is responsible for ensuring that the lifetime of the underlying character buffer extends until it is no longer referenced.

Example 1

Construction from a string parses the input as a *URI-reference* and throws an exception on error. Upon success, the constructed object points to the passed character buffer; ownership is not transferred.

```
url_view u( "https://www.example.com/index.htm?text=none#a1" );
```

Example 2

Parsing functions like `parse_uri_reference` return a `boost::system::result` containing either a valid `url_view` upon success, otherwise they contain an error. The error can be converted to an exception by the caller if desired:

```
system::result< url_view > rv = parse_uri_reference( "https://www.example.com/index.htm?text=none#a1" );
```

BNF

- Parsing
- Components
- Segments
- Params

- Normalization
- String Token
- Percent Encoding
- Formatting

- Parse Rules
- Character Sets
- Compound Rules
- Ranges
- RFC 3986

- QR Code
- Finicky
- mailto URLs
- Magnet Link
- File Router
- Router
- Sanitizing URLs

BNF

```
URI-reference = URI / relative-ref

URI           = scheme ":" hier-part [ "?" query ] [ "#" fragment ]

relative-ref  = relative-part [ "?" query ] [ "#" fragment ]
```

Specification

- [Uniform Resource Identifier \(URI\): Generic Syntax \(rfc3986\)](#)

Base Classes

Name	Description
<code>url_view_base</code>	Common functionality for containers

Member Functions

Name	Description
<code>url_view</code> [constructor]	Constructors
<code>~url_view</code> [destructor]	Destructor
<code>operator=</code>	Assignment
<code>authority</code>	Return the authority
<code>buffer</code>	Return the url string
<code>compare</code>	Return the result of comparing this with another url
<code>data</code>	Return a pointer to the url's character buffer
<code>empty</code>	Return true if the url is empty
<code>encoded_authority</code>	Return the authority.
<code>encoded_fragment</code>	Return the fragment
<code>encoded_host</code>	Return the host
<code>encoded_host_address</code>	Return the host

Mr.Docs

- ▶ [Getting Started](#)
- ▶ [Commands](#)
- ▶ [Configuration](#)
- ▶ [Generators](#)
- ▶ [Extensions](#)
- ▶ [Contribute](#)
- [Design Notes](#)
- [License](#)

ON THIS PAGE

- Synopsis**
- [Description](#)
- [Types](#)
- [Member Functions](#)
- [Static Member Functions](#)
- [Friends](#)
- [Non-Member Functions](#)

MRDOCS::CORPUS

The collection of declarations in extracted form.

SYNOPSIS

Declared in `<mrdocs/Corpus.hpp>`

```
class Corpus;
```

DESCRIPTION

A corpus is a plain value holding the extracted symbols, so it can be moved freely and stays valid wherever it is built. It does not own the configuration: the `Config` that drives a build is passed to `build`, and again to whatever consumes the corpus (for example a `Generator`). Use `build` to produce one from a `Config`.

- ▶ [Getting Started](#)
- ▶ [Commands](#)
- ▶ [Configuration](#)
- ▶ [Generators](#)
- ▶ [Extensions](#)
- ▶ [Contribute](#)
- [Design Notes](#)
- [License](#)

[Synopsis](#)[Description](#)[Types](#)[Member Functions](#)[Static Member Functions](#)[Friends](#)[Non-Member Functions](#)

TYPES

Name	Description
<code>TraverseOptions</code>	Options to traverse the members of a Symbol.
<code>iterator</code>	The iterator type for the index of all symbols.

MEMBER FUNCTIONS

Name	Description
<code>begin</code>	Return the begin iterator for the index of all symbols.
<code>empty</code>	Whether the corpus contains any symbols.
<code>end</code>	Return the end iterator for the index.
<code>exists</code>	Return true if a Symbol with the specified symbol ID exists.
<code>finalize</code>	Finalize the corpus.
<code>find</code>	<code>find</code> overloads
<code>get</code>	Return the Symbol with the specified symbol ID.
<code>globalNamespace</code>	Return the metadata for the global namespace.
<code>lookup</code>	<code>lookup</code> overloads
<code>qualifiedName</code>	<code>qualifiedName</code> overloads

It just understands C++.

terminate.hpp

```
1  /** Exit the program.  
2  
3      The program will end immediately.  
4  
5      @note Functions registered with `std::atexit` are not invoked.  
6  */  
7  [[noreturn]]  
8  void  
9  terminate() noexcept;
```

terminate

Exit the program.

Synopsis

Declared in `<terminate.hpp>`

```
[[noreturn]]  
void  
terminate() noexcept;
```

Description

The program will end immediately.

NOTE

Functions registered with `std::atexit` are not invoked.

Return Value

NOTE

This function does not return.

distance.hpp

```
1  /** Return the distance between two points
2
3      This function returns the distance between two points
4      according to the Euclidean distance formula.
5
6      @param x0 The x-coordinate of the first point
7      @param y0 The y-coordinate of the first point
8      @param x1 The x-coordinate of the second point
9      @param y1 The y-coordinate of the second point
10     @return The distance between the two points
11 */
12 double
13 distance(double x0, double y0, double x1, double y1);
```

distance

Return the distance between two points

Synopsis

Declared in `<distance.hpp>`

```
double
distance(
    double x0,
    double y0,
    double x1,
    double y1);
```

Description

This function returns the distance between two points according to the Euclidean distance formula.

Return Value

The distance between the two points

Parameters

Name	Description
x0	The x-coordinate of the first point
y0	The y-coordinate of the first point
x1	The x-coordinate of the second point
y1	The y-coordinate of the second point

is_prime.hpp

```
1  /** Return true if a number is prime.
2
3  @par Complexity
4
5      Linear in  $n$ .
6
7  @return Whether  $n$  is prime.
8  @param  $n$  The number to test
9  */
10 bool
11 is_prime(unsigned long long n) noexcept;
```

is_prime

Return true if a number is prime.

Synopsis

Declared in `<is_prime.hpp>`

```
bool
is_prime(unsigned long long n) noexcept;
```

Complexity

Linear in n.

Return Value

Whether `n` is prime.

Parameters

Name	Description
<code>n</code>	The number to test

sqrt.hpp

```
1  #include <type_traits>
2
3  /** Computes the integer square root  $\lfloor \sqrt{\text{value}} \rfloor$ .
4
5      This function calculates the square root of a given integral value
6      using bit manipulation, in  $O(\log \text{value})$  time.
7
8      @throws std::invalid_argument if the input value is negative.
9
10     @tparam T The type of the input value. Must be an integral type.
11     @param value The integral value to compute the square root of.
12     @return The square root of the input value.
13  */
14  template <typename T>
15  std::enable_if_t<std::is_integral_v<T>, T> sqrt(T value);
```

sqrt

Computes the integer square root $\lfloor \sqrt{value} \rfloor$.

Synopsis

Declared in `<sqrt.hpp>`

```
template<typename T>
T
sqrt(T value)
requires std::is_integral_v<T>;
```

Description

This function calculates the square root of a given integral value using bit manipulation, in $O(\log value)$ time.

Exceptions

Name	Thrown on
<code>std::invalid_argument</code>	if the input value is negative.

Return Value

The square root of the input value.

Template Parameters

Name	Description
<code>T</code>	The type of the input value. Must be an integral type.

Parameters

function_object.hpp

```
1  struct abs_fn
2  {
3      /** Compute the absolute value.
4
5      @param x The input value.
6      @return The absolute value of x.
7      */
8      double
9      operator()(double x) const noexcept;
10 };
11
12 /** Return the absolute value of a number. */
13 constexpr abs_fn abs = {};
```

abs

Compute the absolute value.

Synopsis

Declared in `<function_object.hpp>`

```
double
abs(double x) noexcept;
```

NOTE

This function is defined as an Algorithm Function Object (AFO).

Return Value

The absolute value of x.

Parameters

Name	Description
x	The input value.

Install

Binary Packages

Binary packages are available from our [Release Page](#). Most users should start here; if no binary release fits your platform, build from source below.

	Windows			Linux		macOS	
Release	7z	msi	zip	tar.xz	tar.gz	tar.xz	tar.gz
master	MrDocs-master-	MrDocs-master-	MrDocs-master-	MrDocs-master-	MrDocs-master-	MrDocs-master-	MrDocs-master-
September 5, 2026	win64.7z	win64.msi	win64.zip	Linux.tar.xz	Linux.tar.gz	Darwin.tar.xz	Darwin.tar.gz
	(89 MB)	(164 MB)	(167 MB)	(43 MB)	(69 MB)	(36 MB)	(60 MB)



terminal

```
1 git clone https://github.com/cppalliance/mrdocs.git
2 cd mrdocs
3 python bootstrap.py --yes
```

terminal

```
1 git clone https://github.com/cppalliance/mrdocs.git
2 cd mrdocs
3 python bootstrap.py --yes
```

docs/mrdocs.yml

```
1 source-root: ..  
2 compilation-database: compile_commands.json
```

terminal

```
1 mrdocs --config=docs/mrdocs.yml
```

docs/mrdocs.yml

```
1 source-root: ..  
2 compilation-database: ../CMakeLists.txt
```

terminal

```
1 mrdocs --config=docs/mrdocs.yml
```

docs/mrdocs.yml

```
1 source-root: ..  
2 input:  
3   - ../include  
4 includes:  
5   - ../include
```

terminal

```
1 mrdocs --config=docs/mrdocs.yml
```

```
include/logr/log.hpp
```

```
1 namespace logr::detail {  
2 struct scope_token { ~scope_token(); };  
3 }  
4 namespace logr {  
5 /// Attach a key/value pair to every log line.  
6 detail::scope_token  
7 scoped_context(char const* key, char const* value);  
8 }
```

```
include/logr/log.hpp
```

```
1 namespace logr::detail {  
2 struct scope_token { ~scope_token(); };  
3 }  
4 namespace logr {  
5 /// Attach a key/value pair to every log line.  
6 detail::scope_token  
7 scoped_context(char const* key, char const* value);  
8 }
```

docs/mrdocs.yml

```
1 implementation-defined:
2   - 'logr::detail::**'
```

logr::scoped_context

Attach a key/value pair to every log line.

Synopsis

Declared in `<include/logr/log.hpp>`

```
/* implementation-defined */  
scoped_context(  
    char const* key,  
    char const* value);
```

```
include/numerics/clamp.hpp
```

```
1 struct clamp_fn
2 {
3     /// Constrain a value to `[lo, hi]'.
4     template <class T>
5     T operator()(T const& v, T const& lo,
6                 T const& hi) const;
7 };
8 inline constexpr clamp_fn clamp = {};
```

include/numerics/clamp.hpp

```
1 struct clamp_fn
2 {
3     /// Constrain a value to `[lo, hi]`.
4     template <class T>
5     T operator()(T const& v, T const& lo,
6                 T const& hi) const;
7 };
8 inline constexpr clamp_fn clamp = {};
```

clamp

Constrain a value to `[lo, hi]`.

Synopsis

Declared in `<include/numerics/clamp.hpp>`

```
template<class T>
T
clamp(
    T const& v,
    T const& lo,
    T const& hi);
```

NOTE

This function is defined as an [Algorithm Function Object \(AFO\)](#).

A struct, its members, and a function, mixing `/**` and `///`

```
1  /** The main information about a person
2
3      This class represents a person with
4      a name and an age. This is the
5      information about a person that
6      we need to store in our system.
7  */
8  struct person
9  {
10     /// The person's full name.
11     char const* name;
12
13     /// The person's age in years.
14     int age;
15 };
16
17 /** A function to greet a person
18
19     This function takes a person and
20     prints a greeting message.
21
22     @param p The person to greet
23 */
24 int greet(person* p)
```

A struct, its members, and a function, mixing `/**` and `///`

```
1  /// The main information about a person
2
3      This class represents a person with
4      a name and an age. This is the
5      information about a person that
6      we need to store in our system.
7  */
8  struct person
9  {
10     /// The person's full name.
11     char const* name;
12
13     /// The person's age in years.
14     int age;
15 };
16
17 /** A function to greet a person
18
19     This function takes a person and
20     prints a greeting message.
21
22     @param p The person to greet
23 */
24 void greet(person const& p);
```

person

The main information about a person

Synopsis

Declared in `<person.cpp>`

```
struct person;
```

Description

This class represents a person with a name and an age. This is the information about a person that we need to store in our system.

Data Members

Name	Description
<code>age</code>	The person's age in years.
<code>name</code>	The person's full name.

Non-Member Functions

Name	Description
<code>greet</code>	A function to greet a person

greet

A function to greet a person

Synopsis

Declared in `<person.cpp>`

```
void
greet(person const& p);
```

Description

This function takes a person and prints a greeting message.

Parameters

Name	Description
p	The person to greet

Paragraphs

```
1  /** Computes the absolute value of an integer.  
2  
3      Returns the input unchanged when non-negative,  
4      otherwise returns its negation.  
5  */  
6  int absolute(int x);
```

absolute

Computes the absolute value of an integer.

Synopsis

Declared in `<paragraph.cpp>`

```
int  
absolute(int x);
```

Description

Returns the input unchanged when non-negative, otherwise returns its negation.

config

A configuration record.

Synopsis

Declared in `<heading.cpp>`

```
struct config;
```

Thread safety

Instances are immutable after construction and may be shared across threads without locking.

Lists

```
1  /** Initializes the connection.  
2  
3      The function performs three steps in order:  
4  
5      - Allocates the socket.  
6      - Performs the handshake.  
7      - Caches the credentials.  
8  */  
9  void connect();
```

connect

Initializes the connection.

Synopsis

Declared in `<list.cpp>`

```
void  
connect();
```

Description

The function performs three steps in order:

- Allocates the socket.
- Performs the handshake.
- Caches the credentials.

Tables

```
1  /** Returns a status code describing the result of `parse`.
2
3      <table>
4          <tr><th>Code</th><th>Meaning</th></tr>
5          <tr><td>0</td><td>Success</td></tr>
6          <tr><td>1</td><td>Syntax error</td></tr>
7          <tr><td>2</td><td>I/O error</td></tr>
8      </table>
9  */
10 int parse_status();
```

parse_status

Returns a status code describing the result of `parse` .

Synopsis

Declared in `<table.cpp>`

```
int
parse_status();
```

Description

Code	Meaning
0	Success
1	Syntax error
2	I/O error

Return Value

a status code describing the result of `parse` .

Code blocks

```
1  /** Computes a hash code for a byte sequence.
2
3  Typical use:
4
5  @code
6  auto h = compute_hash("hello", 5);
7  @endcode
8  */
9  unsigned compute_hash(const char* data, unsigned n);
```

compute_hash

Computes a hash code for a byte sequence.

Synopsis

Declared in `<code-block.cpp>`

```
unsigned int  
compute_hash(  
    char const* data,  
    unsigned int n);
```

Description

Typical use:

```
auto h = compute_hash("hello", 5);
```

Admonitions

```
1  /** Closes the handle.
2
3  @note Closing a handle invalidates any iterators
4         that point into the underlying buffer.
5
6  @warning Do not call this from a destructor that
7           runs during program termination.
8
9  @tip Prefer the RAII wrapper `scoped_handle`,
10         which closes automatically at scope exit.
11  */
12 void close_handle();
```

close_handle

Closes the handle.

Synopsis

Declared in `<admonitions.cpp>`

```
void  
close_handle();
```

Description

NOTE

Closing a handle invalidates any iterators that point into the underlying buffer.

WARNING

Do not call this from a destructor that runs during program termination.

TIP

Prefer the RAII wrapper `scoped_handle`, which closes automatically at scope exit.

Parameters, template parameters, and return value together

```
1  /** Resamples an audio signal to a new sample rate.
2
3      @tparam Sample The sample type, usually `float` or `short`.
4      @param[in] in      The input samples.
5      @param[in] count Number of input samples.
6      @param[in] ratio Output rate divided by input rate.
7      @param[out] out    Buffer that receives the resampled signal.
8      @returns The number of samples written to `out`.
9  */
10 template <class Sample>
11 unsigned resample(Sample const* in, unsigned count, double ratio, Sample* out);
```

resample

Resamples an audio signal to a new sample rate.

Synopsis

Declared in `<resample.cpp>`

```
template<class Sample>
unsigned int
resample(
    Sample const* in,
    unsigned int count,
    double ratio,
    Sample* out);
```

Return Value

The number of samples written to `out`.

Template Parameters

Name	Description
Sample	The sample type, usually <code>float</code> or <code>short</code> .

Parameters

Name	Description
in [in]	The input samples.
count [in]	Number of input samples.
ratio [in]	Output rate divided by input rate.

Preconditions, postconditions, and exceptions together

```
1  /** Moves funds from one account to another.
2
3      @pre `amount` is positive.
4      @pre `from` and `to` are different accounts.
5      @post `from` falls by `amount` and `to` rises by the same.
6      @throws std::invalid_argument If an account number is unknown.
7      @throws std::out_of_range If `from` cannot cover `amount`.
8  */
9  void transfer(int from, int to, long amount);
```

transfer

Moves funds from one account to another.

Synopsis

Declared in `<transfer.cpp>`

```
void
transfer(
    int from,
    int to,
    long amount);
```

Exceptions

Name	Thrown on
<code>std::invalid_argument</code>	If an account number is unknown.
<code>std::out_of_range</code>	If <code>from</code> cannot cover <code>amount</code> .

Preconditions

- `amount` is positive.
- `from` and `to` are different accounts.

Postconditions

- `from` falls by `amount` and `to` rises by the same.

Relates

```
1  /** A pixel record. */
2  struct pixel { int x, y; };
3
4  /** Computes the Euclidean distance between two pixels.
5  */
6  @relates pixel
7  */
8  double distance(pixel const& a, pixel const& b);
```

pixel

A pixel record.

Synopsis

Declared in `<relates.cpp>`

```
struct pixel;
```

Data Members

Name
<code>x</code>
<code>y</code>

Non-Member Functions

Name	Description
<code>distance</code>	Computes the Euclidean distance between two pixels.

Symbol flags

```
1 namespace detail {
2     /** The compiled bytecode the matcher executes.
3
4     @implementationdefined
5     */
6     struct program;
7 }
8
9 /** A compiled regular expression.
10
11     Holds the program built from a pattern and tests it against input
12     text. Create one with `compile` rather than constructing it directly.
13
14     @seebelow
15     */
16 class regex
17 {
18     detail::program* prog_;
19 public:
20     bool match(char const* text) const;
21 };
22
23 struct compile_fn
24 {
```

Symbol flags

```
3
4     @implementationdefined
5     */
6 struct program;
7 }
8
9  /** A compiled regular expression.
10
11     Holds the program built from a pattern and tests it against input
12     text. Create one with `compile` rather than constructing it directly.
13
14     @seebelow
15     */
16 class regex
17 {
18     detail::program* prog_;
19 public:
20     bool match(char const* text) const;
21 };
22
23 struct compile_fn
24 {
25     /** Compiles `pattern` into a regular expression. */
26     regex operator()(char const* pattern) const;
27 }
```

Symbol flags

```
18 detail::program prog_7
19 public:
20     bool match(char const* text) const;
21 };
22
23 struct compile_fn
24 {
25     /** Compiles `pattern` into a regular expression. */
26     regex operator()(char const* pattern) const;
27
28     int syntax; // extra member: auto-detection skips this type
29 };
30
31 /** @functionobject
32
33     The extra `syntax` member defeats auto-detection.
34 */
35 constexpr compile_fn compile = {};
36
37 /** Exposes a regex's compiled bytecode, for debugging.
38
39     @returns The bytecode, in an implementation-defined form.
40 */
41 detail::program const& bytecode(regex const& re);
```

regex

A compiled regular expression.

Synopsis

Declared in `<regex.cpp>`

```
class regex { /* see-below */ };
```

Description

Holds the program built from a pattern and tests it against input text. Create one with `compile` rather than constructing it directly.

bytecode

Exposes a regex's compiled bytecode, for debugging.

Synopsis

Declared in `<regex.cpp>`

```
/* implementation-defined */ const&  
bytecode(regex const& re);
```

Return Value

The bytecode, in an implementation-defined form.

Parameters

Name	Description
re	A compiled regular expression.

compile

Compiles `pattern` into a regular expression.

Synopsis

Declared in `<regex.cpp>`

```
regex  
compile(char const* pattern);
```

NOTE

This function is defined as an Algorithm Function Object (AFO).

Return Value

A compiled regular expression.

format_money

Formats a quantity for **human** reading.

Synopsis

Declared in `<text-formatting.cpp>`

```
char const*  
format_money(long cents);
```

Description

Returns a **localized** string. Pass the raw value in `cents`; the ~~deprecated~~ `format_dollars` spelling is gone. Out-of-range inputs are **flagged** in the output.

Links

```
1 /** A logger compatible with the
2     [SLF4J specification](https://www.slf4j.org).
3 */
4 struct logger { };
```

logger

A logger compatible with the [SLF4J specification](#).

Synopsis

Declared in `<link.cpp>`

```
struct logger;
```

Cross-references

```
1  /** Triples the input. */
2  int triple(int x);
3
4  /** Doubles the input.
5
6          Closely related to @ref triple.
7   */
8  int twice(int x);
```

triple

Triples the input.

Synopsis

Declared in `<cross-reference.cpp>`

```
int  
triple(int x);
```

twice

Doubles the input.

Synopsis

Declared in `<cross-reference.cpp>`

```
int  
twice(int x);
```

Description

Closely related to `triple`.

docs/mrdocs.yml

```
1 source-root: ..  
2 compilation-database: ../CMakeLists.txt
```

docs/mrdocs.yml

```
1 source-root: ..  
2 compilation-database: ../CMakeLists.txt  
3 generator: html  
4 include-symbols:  
5   - 'mylib:**'
```

```
1 mrdocs --config=path/to/mrdocs.yml --output=../docs/reference
```

docs/mrdocs.yml

```
1 source-root: ..  
2 input:  
3   - ../include  
4 includes:  
5   - ../external  
6 defines:  
7   - LIB_BUILD
```

mrdocs.yml

```
1 include-symbols:
2   - 'jzon:**'
3 exclude-symbols:
4   - 'jzon::unsafe_*'
5   - 'jzon::extra:**'
```

```
include/jzon/parser.hpp
```

```
1 namespace jzon {  
2  
3     /// A parsed JSON document.  
4     class value;  
5  
6     /** Validate that the document is well-formed JSON.  
7  
8         Parses `source` without constructing a value tree, reporting only  
9         whether the input conforms to RFC 8259.  
10  
11         @param source A null-terminated UTF-8 JSON document.  
12         @return `true` if the document is syntactically valid JSON.  
13     */  
14     bool validate(char const* source);  
15  
16     /** Parse a JSON document.  
17  
18         Validates `source` and returns the resulting value tree.  
19  
20         @param source A null-terminated UTF-8 JSON document.  
21     */  
22     value parse(char const* source);  
23  
24     /** Parse a JSON document from a file with the following signature:
```

```
include/jzon/parser.hpp
```

```
14 bool validate(char const* source);
15
16 /** Parse a JSON document.
17
18 Validates `source` and returns the resulting value tree.
19
20 @param source A null-terminated UTF-8 JSON document.
21 */
22 value parse(char const* source);
23
24 /** Parse a JSON document without validating.
25
26 Assumes the caller has already verified `source` with @ref validate.
27 The behavior is undefined if `source` is not well-formed JSON.
28 */
29 value unsafe_parse(char const* source);
30
31 /** Count the number of top-level elements in `source`.
32
33 Assumes the caller has already verified `source` with @ref validate.
34 */
35 unsigned long unsafe_element_count(char const* source);
36
37 }
```

```
include/jzon/extra/utilities.hpp
```

```
1 namespace jzon::extra {  
2  
3   /// Turn on parser tracing for the current thread.  
4   void enable_trace();  
5  
6   /// Turn off parser tracing for the current thread.  
7   void disable_trace();  
8  
9 }
```

mrdocs.yml

```
1 include-macros:
2   - 'MY_LIB_*'
3 exclude-macros:
4   - 'MY_LIB_DETAIL_*'
5   - '*_IMPL'
```

Example

```
1  /// The public assertion macro.
2  #define MY_LIB_ASSERT(x) ((x) ? (void)0 : ::my_lib::fail())
3
4  /// An internal helper, dropped by the exclude pattern.
5  #define MY_LIB_DETAIL_CHECK(x) ((void)(x))
6
7  /// The implementation behind MY_LIB_ASSERT, dropped by the exclude pattern.
8  #define MY_LIB_ASSERT_IMPL(x) ((void)(x))
```

MY_LIB_ASSERT

The public assertion macro.

Synopsis

Declared in `<exclude-macros.cpp>`

```
#define MY_LIB_ASSERT(x)
```

Example

```
1  /// A 2D vector.
2  struct vec2
3  {
4      /// Construct from `x` and `y` components.
5      vec2(double x, double y);
6
7      vec2(vec2 const&);
8      vec2(vec2&&) noexcept;
9      ~vec2();
10
11     vec2& operator=(vec2 const&);
12
13     bool operator==(vec2 const&) const;
14
15     /// Returns the Euclidean length of the vector.
16     double magnitude() const;
17
18     /// Translate this vector by `delta`.
19     vec2 translated(vec2 const& delta) const;
20 };
```


vec2

A 2D vector.

Synopsis

Declared in `<auto-function-metadata.cpp>`

```
struct vec2;
```

Member Functions

Name	Description
<code>vec2</code> [constructor]	Construct from <code>x</code> and <code>y</code> components.
<code>~vec2</code> [destructor]	Destructor
<code>operator=</code>	Copy assignment operator
<code>magnitude</code>	Returns the Euclidean length of the vector.
<code>translated</code>	Translate this vector by <code>delta</code> .
<code>operator==</code>	Equality operator

vec2::vec2

Construct from `x` and `y` components.

Synopses

Declared in `<auto-function-metadata.cpp>`

Copy constructor

```
vec2(vec2 const& other) {
```

Example

```
1 namespace detail {
2
3     /// CRTP base that lifts `compare()` into `operator==` and `operator!=`.
4     template <class T>
5     struct comparable
6     {
7         /// True if `compare()` is `0`.
8         bool operator==(T const& other) const noexcept;
9         /// True otherwise.
10        bool operator!=(T const& other) const noexcept;
11    };
12
13 }
14
15 /// A semantic version `(major, minor)`.
16 struct version : detail::comparable<version>
17 {
18     /// Construct from major and minor numbers.
19     version(int major, int minor) noexcept;
20
21     /// Three-way comparison against another version.
22     int compare(version const& other) const noexcept;
23
24     /// True if this version is greater than the other.
```

Example

```
5 // ... comparable
6 {
7     /// True if `compare()` is `0`.
8     bool operator==(T const& other) const noexcept;
9     /// True otherwise.
10    bool operator!=(T const& other) const noexcept;
11 };
12
13 }
14
15 /// A semantic version `(major, minor)`.
16 struct version : detail::comparable<version>
17 {
18     /// Construct from major and minor numbers.
19     version(int major, int minor) noexcept;
20
21     /// Three-way comparison against another version.
22     int compare(version const& other) const noexcept;
23
24     /// The major component.
25     int major;
26     /// The minor component.
27     int minor;
28 };
```

mrdocs.yml

```
1 inherit-base-members: copy-dependencies
2 implementation-defined:
3   - 'detail:**'
```

version

A semantic version (major, minor).

Synopsis

Declared in <inherited-members-specialization.cpp>

```
struct version
    : /* implementation-defined */
```

Base Classes

Name	Description
<code>/* implementation-defined */</code>	CRTP base that lifts <code>compare()</code> into <code>operator==</code> and <code>operator!=</code> .

Member Functions

Name	Description
<code>version</code> [constructor]	Construct from major and minor numbers.
<code>compare</code>	Three-way comparison against another version.
<code>operator==</code>	True if <code>compare()</code> is 0.
<code>operator!=</code>	True otherwise.

Data Members

Name	Description
<code>major</code>	The major component.
<code>minor</code>	The minor component.

version::operator==

True if `compare()` is 0.

Synopsis

Declared in `<inherited-members-specialization.cpp>`

```
bool
operator==(version const& other) const noexcept;
```

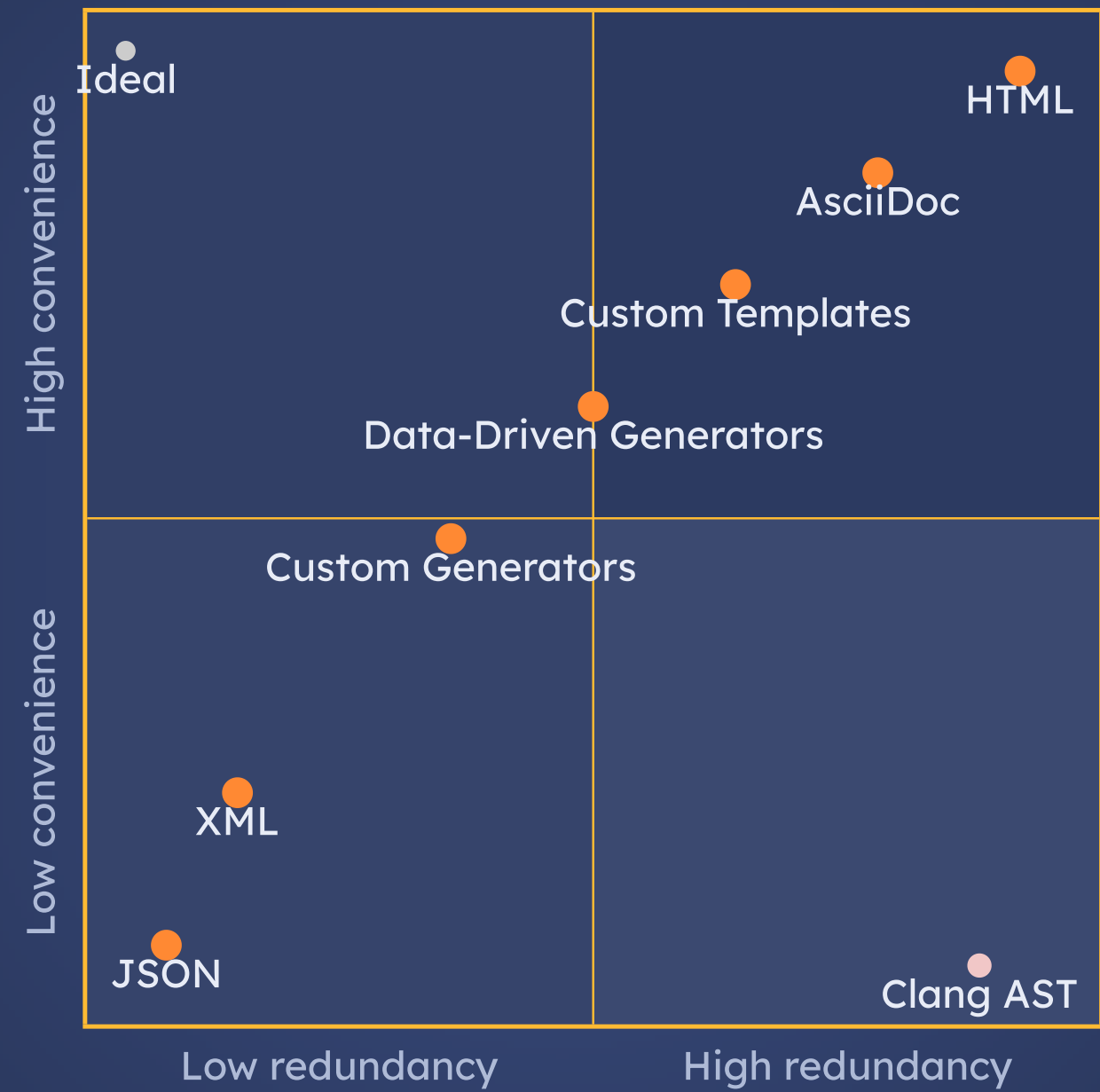
Return Value

`true` if the objects are equal, `false` otherwise

Parameters

Name	Description
<code>other</code>	The right operand

Generators



```
1 generator: noop
```

mrdocs.yml

```
1  # A validation-only MrDocs run: extract everything, write nothing, and fail on
2  # any documentation problem. Point CI at this config to gate the docs.
3  generator: noop
4  warnings: true
5  warn-as-error: true
6  warn-if-undocumented: true
7  warn-no-paramdoc: true
8  source-root: include
9  input:
10    - include
```

include/geo/area.hpp

```
1  #ifndef GEO_AREA_HPP
2  #define GEO_AREA_HPP
3
4  /// Area calculations for simple shapes.
5  namespace geo {
6
7  /** The area of a rectangle.
8
9      @return The area, `width * height`.
10 */
11 double rectangle_area(double width, double height);
12
13 }
14
15 #endif
```

report.txt

```
1 geo::rectangle_area: Missing documentation for parameter 'width'  
2 geo::rectangle_area: Missing documentation for parameter 'height'
```

CMakeLists.txt

```
1 find_package(mrdocs REQUIRED)
2 add_test(
3     NAME documentation-check
4     COMMAND ${MRDOCS_EXECUTABLE}
5         --config=${CMAKE_CURRENT_SOURCE_DIR}/mrdocs.yml
6         --generator=noop --warn-as-error)
```


fix-docs.py

```
31 LOCATION = re.compile(r"^\s.*:\d+:\d+:\s*$")
32 FOOTER = re.compile(
33     r"^(An issue occurred|If you believe|\s*MrDocs Version|\s*Error Location|"
34     r"\s*Reported From|with the following details|"
35     r"max-errors=|error limit reached)")
36
37
38 def main():
39     """Check, repair, repeat until the docs are clean or the rounds run out."""
40     print(f"Checking with: {shlex.join(CMD)}")
41     timings = []
42     for round in range(1, MAX_ROUNDS + 1):
43         clean, report, check_time = check()
44         if clean:
45             print(f"Documentation is clean after {round - 1} round(s).")
46             return summary(timings, 0)
47         # No parseable diagnostics (a compile error or a broken config) is
48         # not "clean" and not "stalled": hand the agent the raw output so it
49         # can unblock extraction, and keep going.
50         batch = diagnostics(report)[:BATCH] or [report]
51         if stalled(batch):
52             print("The same locations keep coming back; stopping so a human "
53                 "can look.", file=sys.stderr)
54             return summary(timings, 1)
55         print(f"Round {round}: check took {check_time: .0f}s: "
```

fix-docs.py

```
40     print(f"Checking with: {shlex.join(CMD)}")
41     timings = []
42     for round in range(1, MAX_ROUNDS + 1):
43         clean, report, check_time = check()
44         if clean:
45             print(f"Documentation is clean after {round - 1} round(s).")
46             return summary(timings, 0)
47         # No parseable diagnostics (a compile error or a broken config) is
48         # not "clean" and not "stalled": hand the agent the raw output so it
49         # can unblock extraction, and keep going.
50         batch = diagnostics(report)[:BATCH] or [report]
51         if stalled(batch):
52             print("The same locations keep coming back; stopping so a human "
53                 "can look.", file=sys.stderr)
54             return summary(timings, 1)
55         print(f"Round {round}: check took {check_time:.0f}s; "
56             f"handing {len(batch)} location(s) to the agent.")
57         agent_time = repair(batch)
58         print(f"Round {round}: agent took {agent_time:.0f}s.")
59         timings.append((check_time, agent_time))
60     print(f"Still failing after {MAX_ROUNDS} rounds; stopping so a human "
61         f"can look.", file=sys.stderr)
62     return summary(timings, 1)
63
64
```

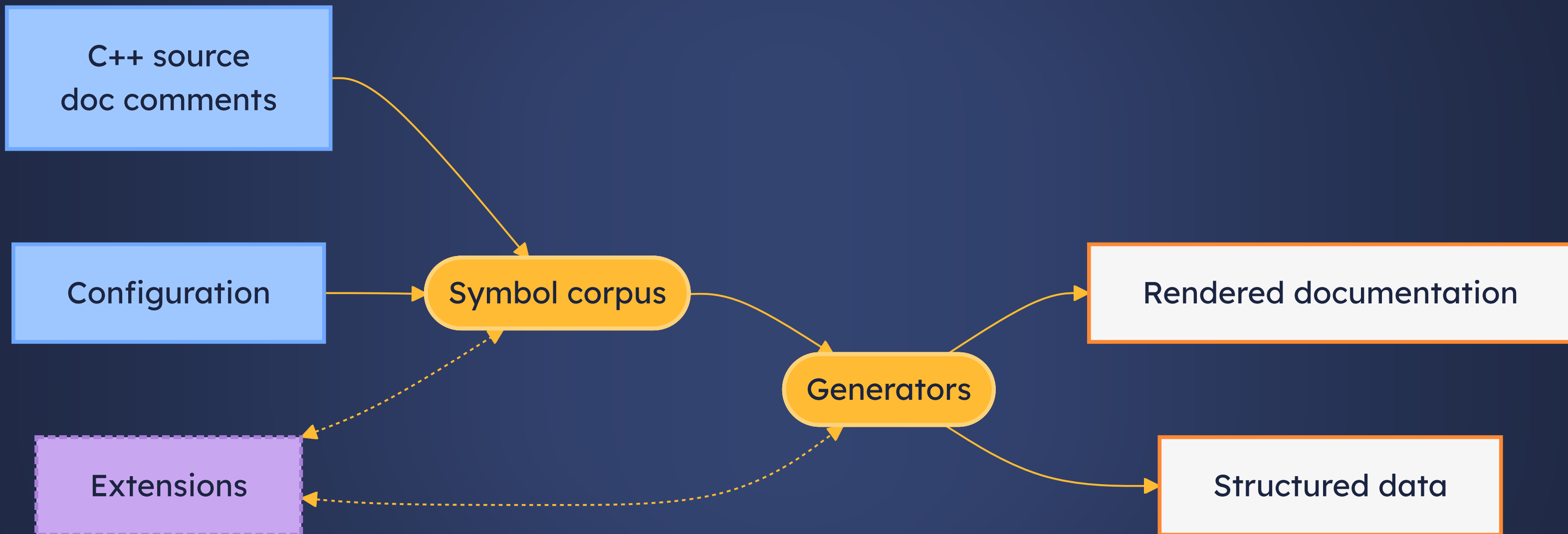
fix-docs.py

```
45         print(f"Documentation is clean after {round - 1} round(s).")
46         return summary(timings, 0)
47         # No parseable diagnostics (a compile error or a broken config) is
48         # not "clean" and not "stalled": hand the agent the raw output so it
49         # can unblock extraction, and keep going.
50         batch = diagnostics(report)[:BATCH] or [report]
51         if stalled(batch):
52             print("The same locations keep coming back; stopping so a human "
53                 "can look.", file=sys.stderr)
54             return summary(timings, 1)
55         print(f"Round {round}: check took {check_time:.0f}s; "
56             f"handing {len(batch)} location(s) to the agent.")
57         agent_time = repair(batch)
58         print(f"Round {round}: agent took {agent_time:.0f}s.")
59         timings.append((check_time, agent_time))
60         print(f"Still failing after {MAX_ROUNDS} rounds; stopping so a human "
61             f"can look.", file=sys.stderr)
62         return summary(timings, 1)
63
64
65     def check():
66         """Run the strict no-op check; return (clean?, report, seconds).
67
68         Clean requires both a success exit code and a non-empty corpus: a run
69         that extracts zero declarations is a broken configuration, not a clean
```

fix-docs.py

```
30 ANSI = re.compile(r"\x1b\[([0-9;]*)m")
31 LOCATION = re.compile(r"^\.S.*:\d+:\d+:\s*$")
32 FOOTER = re.compile(
33     r"^(An issue occurred|If you believe|\s*MrDocs Version|\s*Error Location|"
34     r"\s*Reported From|with the following details|"
35     r"max-errors=|error limit reached)")
36
37
38 def main():
39     """Check, repair, repeat until the docs are clean or the rounds run out."""
40     print(f"Checking with: {shlex.join(CMD)}")
41     timings = []
42     for round in range(1, MAX_ROUNDS + 1):
43         clean, report, check_time = check()
44         if clean:
45             print(f"Documentation is clean after {round - 1} round(s).")
46             return summary(timings, 0)
47         # No parseable diagnostics (a compile error or a broken config) is
48         # not "clean" and not "stalled": hand the agent the raw output so it
49         # can unblock extraction, and keep going.
50         batch = diagnostics(report)[:BATCH] or [report]
51         if stalled(batch):
52             print("The same locations keep coming back; stopping so a human "
53                 "can look.", file=sys.stderr)
54         return summary(timings, 1)
```

Extensions



custom-addons/generator/html/partials/markup/code-block.html.hbs

```
1 {{!-- Override of share/mrdocs/addons/generator/html/partials/markup/code-block.html.hbs --}}
2 <div class="code-block-wrapper">
3   <button class="copy-button" type="button" aria-label="Copy code">Copy</button>
4   <pre><code class="source-code cpp">{{> @partial-block }}</code></pre>
5 </div>
```

mrdocs.yml

```
1 addons-supplemental:  
2   - custom-addons
```

override-code-block.cpp

```
1 /// Square `value` and return the result.  
2 double square(double value);
```

square

Square `value` and return the result.

Synopsis

Declared in `<override-code-block.cpp>`

```
double  
square(double value);
```

COPY

addons/generator/common/partial/symbol.hbs

```
1  {{!--
2      Project override: render the synopsis last, after the description, so the
3      prose and examples come first on the page. Only the position of
4      `symbol/section/synopsis` changes from the base template.
5  --}}
6  {{#> markup/section name="symbol"}}
7  {{> symbol/section/header}}
8  {{> symbol/section/attribute-admonitions ~}}
9  {{> symbol/section/function-object-admonition}}
10 {{> symbol/section/description}}
11 {{> symbol/section/base-classes}}
12 {{> symbol/section/protected-base-classes}}
13 {{> symbol/section/members}}
14 {{> symbol/section/specializations}}
15 {{> symbol/section/deduction-guides}}
16 {{> symbol/section/friends}}
17 {{> symbol/section/using-directives}}
18 {{> symbol/section/non-member-functions}}
19 {{> symbol/section/derived-classes}}
20 {{> symbol/section/introduced-symbols}}
21 {{> symbol/section/noexcept-specification}}
22 {{> symbol/section/exceptions}}
23 {{> symbol/section/return-value}}
24 {{> symbol/section/synopsis}}
```

addons/generator/common/partials/symbol/signature/see-below.hbs

```
1 {{ str '/*' }} see above {{ str '*/' }}
```

override-see-below.cpp

```
1 namespace net {
2
3   /** A move-only handle to an open TCP connection.
4
5       Obtain one from `connect`; it closes the socket when it goes out of scope.
6       Send and receive through the free functions in this namespace:
7
8       @code
9       auto c = net::connect("example.org", 80);
10      net::write(c, request);
11      @endcode
12
13      Its representation is an implementation detail that changes between releases,
14      so the definition is not shown here; use it only through the operations
15      described above.
16
17      @seebelow
18   */
19   class connection
20   {
21       int fd_;
22   public:
23       /// Close the connection.
24       ~connection() {
```

net::connection

A move-only handle to an open TCP connection.

Description

Obtain one from `connect` ; it closes the socket when it goes out of scope. Send and receive through the free functions in this namespace:

```
auto c = net::connect("example.org", 80);
net::write(c, request);
```

Its representation is an implementation detail that changes between releases, so the definition is not shown here; use it only through the operations described above.

Synopsis

Declared in `<override-see-below.cpp>`

```
class connection { /* see above */ };
```

addons/generator/common/helpers/slug.js

```
1  // A URL-safe slug: lowercased, runs of non-alphanumeric characters collapsed
2  // to a single '-', and leading and trailing dashes removed.
3  function slug(name)
4  {
5      if (!name) {
6          return '';
7      }
8      return String(name)
9          .toLowerCase()
10         .replace(/^[^a-z0-9]+/g, '-')
11         .replace(/^-+|-+$/g, '');
12 }
```

addons/generator/common/partials/location/source.hbs

```
1 {{!--
2     Override of share/mrdocs/addons/generator/common/partials/location/source.hbs.
3     Appends a permalink-style anchor after "Declared in", built from
4     the file path via the `slug` helper.
5 --}}
6 Declared in {{#>markup/code~}}
7     {{ str "<" }}{{ dcl.shortPath }}{{ str ">" }}
8     {{~/markup/code}}{{ str " " }}(anchor: {{#>markup/code}}{{ slug dcl.shortPath }}{{/markup/code}})
```

helper-slug.cpp

```
1 /// Compute the Euclidean distance between two points.  
2 double distance(double x1, double y1, double x2, double y2);
```

distance

Compute the Euclidean distance between two points.

Synopsis

Declared in `<helper-slug.cpp>` (anchor: `helper-slug-cpp`)

```
double
distance(
    double x1,
    double y1,
    double x2,
    double y2);
```

addons/generator/jsonl/mrdocs-generator.yml

```
1 escape:  
2   '"': '\\"'  
3   '\\': '\\\\'
```

addons/generator/jsonl/layouts/index.jsonl.hbs

```
1 {{#unless (eq symbol.kind "namespace")~}}
2 {"name": "{{>symbol/qualified-name-text symbol}}", "kind": "{{symbol.kind}}", "anchor": "{{symbol.anchor}}", "brief": "
  {{#if symbol.doc.brief.children}}{{#each symbol.doc.brief.children}}{{literal}}{{/each}}{{/if}}"
3 {{/unless}}
```

simple.cpp

```
1  /// A small two-dimensional point.
2  struct Point
3  {
4      /// Distance from the origin.
5      double length() const;
6
7      /// Translate by an offset.
8      void translate(int dx, int dy);
9  };
```

```
1 generator: jsonl
```

```
1 mrdocs --config=path/to/mrdocs.yml --generator=jsonl
```

simple.jsonl

```
1 {"name": "Point", "kind": "record", "anchor": "Point", "brief": "A small two-dimensional point."}
2 {"name": "Point::length", "kind": "function", "anchor": "Point-length", "brief": "Distance from the origin."}
3 {"name": "Point::translate", "kind": "function", "anchor": "Point-translate", "brief": "Translate by an offset."}
```

addons/generator/md/mrdocs-generator.yml

```
1 extends: html
2 escape:
3   '\': '\\ '
4   '_': '\_ '
5   '*': '\\*'
6   '`': '\\` '
7   '[': '\\['
8   ']': '\\]'
```

addons/generator/tex/mrdocs-generator.yml

```
1 extends: md
2 escape:
3   '\': '\textbackslash{'
4   '_': '\_'
5   '&': '\&'
6   '%': '\%'
7   '#': '\#'
8   '$': '\$'
```


addons/generator/tex/layouts/wrapper.tex.hbs

```
1 \documentclass{article}
2 \title{Reference}
3 \begin{document}
4 \maketitle
5 {{{contents}}}
6 \vfill
7 \hrule
8 \vspace{0.5em}
9 \emph{Created with MrDocs (https://www.mrdocs.com)}
10 \end{document}
```

addons/generator/md/partials/markup/strong.md.hbs

```
1 **{{> @partial-block }}**
```

addons/generator/tex/partials/markup/strong.tex.hbs

```
1 \textbf{ {{~> @partial-block ~}} }
```

simple.cpp

```
1  /// A small two-dimensional point.
2  struct Point
3  {
4      /// Distance from the origin.
5      double length() const;
6
7      /// Translate by an offset.
8      void translate(int dx, int dy);
9  };
```

simple.md

```
1  # Reference
2
3  ## Point
4
5  A small two-dimensional point.
6
7  ### Synopsis
8
9  Declared in `<simple.cpp>`
10
11  ```cpp
12  struct Point;
13  ```
14
15  ### Member Functions
16
17  | Name | Description |
18  | --- | --- |
19  | [`length`](#Point-length) | Distance from the origin. |
20  | [`translate`](#Point-translate) | Translate by an offset. |
21
22  ## [Point](#Point)::length
23
24  Distance from the origin.
```

Reference

Point

A small two-dimensional point.

Synopsis

Declared in `<simple.cpp>`

```
struct Point;
```

Copy icon

Member Functions

Name	Description
<code>length</code>	Distance from the origin.
<code>translate</code>	Translate by an offset.

Point::length

Distance from the origin.

Synopsis

simple.tex

```
1 \documentclass{article}
2 \title{Reference}
3 \begin{document}
4 \maketitle
5 \section{Point}
6
7 A small two-dimensional point.
8
9 \subsection{Synopsis}
10
11 Declared in \texttt{<simple.cpp>}
12
13 \begin{verbatim}
14 struct Point;
15 \end{verbatim}
16
17 \subsection{Member Functions}
18
19 \begin{tabular}{ll}
20 \textbf{Name} & \textbf{Description} \\
21 \hline
22 \texttt{length} & Distance from the origin. \\
23 \texttt{translate} & Translate by an offset. \\
24 \texttt{...}
```

Reference

September 12, 2026

1 Point

A small two-dimensional point.

1.1 Synopsis

Declared in `<simple.cpp>`

```
struct Point;
```

1.2 Member Functions

Name	Description
<code>length</code>	Distance from the origin.
<code>translate</code>	Translate by an offset.

2 Point::length

Distance from the origin.

2.1 Synopsis

Declared in `<simple.cpp>`

```
double
length() const;
```

3 Point::translate

Translate by an offset.

Corpus Transforms

brief-from-name.cpp

```
1 bool is_prime(int n);  
2 bool is_palindrome(char const* s);  
3 bool is_empty(char const* s);
```

addons/extensions/brief_from_name.js

```
1  mrdocs.register_transform("brief-from-name", function(ctx) {
2      for (let i = 0; i < ctx.corpus.symbols.length; ++i) {
3          fillPredicateDoc(ctx.corpus.symbols[i]);
4      }
5  });
6
7  /**
8   * Fill in the brief and parameter doc of an `is_*` predicate from its name.
9   * The subject is the name after `is_` with underscores turned to spaces, so
10  * `is_prime` becomes "Returns true if prime." Does nothing for a symbol that is
11  * not an `is_*` function, and never overwrites a field an author already wrote.
12  * @param {object} sym - The symbol to document, mutated in place.
13  */
14  function fillPredicateDoc(sym) {
15      if (sym.kind !== "function" || sym.name.indexOf("is_") !== 0) {
16          return;
17      }
18      if (!sym.doc) { sym.doc = {}; }
19
20      const subject = sym.name.slice(3).replace(/_/g, " ");
21
22      if (!sym.doc.brief) {
23          sym.doc.brief = "Returns true if " + subject + ".";
24      }
25  }
```

addons/extensions/brief_from_name.js

```
4     }
5   });
6
7   /**
8    * Fill in the brief and parameter doc of an `is_` predicate from its name.
9    * The subject is the name after `is_` with underscores turned to spaces, so
10   * `is_prime` becomes "Returns true if prime." Does nothing for a symbol that is
11   * not an `is_` function, and never overwrites a field an author already wrote.
12   * @param {object} sym - The symbol to document, mutated in place.
13   */
14   function fillPredicateDoc(sym) {
15     if (sym.kind !== "function" || sym.name.indexOf("is_") !== 0) {
16       return;
17     }
18     if (!sym.doc) { sym.doc = {}; }
19
20     const subject = sym.name.slice(3).replace(/_/g, " ");
21
22     if (!sym.doc.brief) {
23       sym.doc.brief = "Returns true if " + subject + ".";
24     }
25
26     if (sym.params.length === 1
27         && (!sym.doc.params || sym.doc.params.length === 0)) {
28       sym.doc.params = [{
```

addons/extensions/brief_from_name.js

```
10  * is_prime becomes "Returns true if prime." Does nothing for a symbol that is
11  * not an 'is_*' function, and never overwrites a field an author already wrote.
12  * @param {object} sym - The symbol to document, mutated in place.
13  */
14  function fillPredicateDoc(sym) {
15      if (sym.kind !== "function" || sym.name.indexOf("is_") !== 0) {
16          return;
17      }
18      if (!sym.doc) { sym.doc = {}; }
19
20      const subject = sym.name.slice(3).replace(/_/g, " ");
21
22      if (!sym.doc.brief) {
23          sym.doc.brief = "Returns true if " + subject + ".";
24      }
25
26      if (sym.params.length === 1
27          && (!sym.doc.params || sym.doc.params.length === 0)) {
28          sym.doc.params = [{
29              name: sym.params[0].name,
30              children: "The input examined for the " + subject + " property."
31          }];
32      }
33  }
```

addons/extensions/brief_from_name.js

```
11  * not an `is_` function, and never overwrites a field an author already wrote.
12  * @param {object} sym - The symbol to document, mutated in place.
13  */
14  function fillPredicateDoc(sym) {
15      if (sym.kind !== "function" || sym.name.indexOf("is_") !== 0) {
16          return;
17      }
18      if (!sym.doc) { sym.doc = {}; }
19
20      const subject = sym.name.slice(3).replace(/_/g, " ");
21
22      if (!sym.doc.brief) {
23          sym.doc.brief = "Returns true if " + subject + ".";
24      }
25
26      if (sym.params.length === 1
27          && (!sym.doc.params || sym.doc.params.length === 0)) {
28          sym.doc.params = [{
29              name: sym.params[0].name,
30              children: "The input examined for the " + subject + " property."
31          }];
32      }
33  }
```

addons/extensions/brief_from_name.lua

```
1  local fillPredicateDoc
2
3  mrdocs.register_transform("brief-from-name", function(ctx)
4      for _, sym in ipairs(ctx.corpus.symbols) do
5          fillPredicateDoc(sym)
6      end
7  end)
8
9  --- Fill in the brief and parameter doc of an `is_*` predicate from its name.
10 --- The subject is the name after `is_` with underscores turned to spaces, so
11 --- `is_prime` becomes "Returns true if prime." Does nothing for a symbol that
12 --- is not an `is_*` function, and never overwrites a field an author already
13 --- wrote.
14 --- @param sym table The symbol to document, mutated in place.
15 function fillPredicateDoc(sym)
16     if sym.kind ~= "function" or sym.name:sub(1, 3) ~= "is_" then
17         return
18     end
19     if not sym.doc then sym.doc = {} end
20
21     local subject = sym.name:sub(4):gsub("_", " ")
22
23     if not sym.doc.brief then
24         sym.doc.brief = "Returns true if " .. subject .. "."
25     end
26 end
```

is_empty

Returns true if empty.

Synopsis

Declared in `<brief-from-name.cpp>`

```
bool  
is_empty(char const* s);
```

Parameters

Name	Description
s	The input examined for the empty property.

is_palindrome

Returns true if palindrome.

Synopsis

Declared in <brief-from-name.cpp>

```
bool
is_palindrome(char const* s);
```

Parameters

Name	Description
s	The input examined for the palindrome property.

is_prime

Returns true if prime.

Synopsis

Declared in <brief-from-name.cpp>

```
bool
is_prime(int n);
```

Parameters

Name	Description
n	The input examined for the prime property.

parse-format-relates.cpp

```
1  /// An HTTP request as a structured value.
2  struct request;
3
4  /// Parse `text` into a request. Returns a valid request on success.
5  request parse_request(char const* text);
6
7  /// Format `r` as the wire-format text of an HTTP request.
8  char const* format_request(request const& r);
9
10 /// A user record.
11 struct user;
12
13 /// Parse `text` into a user.
14 user parse_user(char const* text);
15
16 /// Format `u` as the canonical wire-format text of a user.
17 char const* format_user(user const& u);
```

addons/extensions/parse_format_relates.js

```
1  mrdocs.register_transform("parse-format-relates", function(ctx) {
2      for (let i = 0; i < ctx.corpus.symbols.length; ++i) {
3          const s = ctx.corpus.symbols[i];
4          if (s.kind === "function") {
5              const pname = partnerName(s.name);
6              const partner = pname ? ctx.corpus.lookup(pname) : null;
7              if (partner) {
8                  s.doc = {
9                      sees: [{
10                         children: [{
11                             kind: "reference",
12                             literal: pname,
13                             id: partner.id
14                         }]
15                     }]
16                 };
17             }
18         }
19     }
20 });
21
22 /**
23  * The name of a function's symmetric IO partner: `format_X` for a `parse_X`,
24  * and `parse_X` for a `format_X`. Returns null if no partner is found.
```

addons/extensions/parse_format_relates.js

```
1  mrdocs.register_transform("parse-format-relates", function(ctx) {
2      for (let i = 0; i < ctx.corpus.symbols.length; ++i) {
3          const s = ctx.corpus.symbols[i];
4          if (s.kind === "function") {
5              const pname = partnerName(s.name);
6              const partner = pname ? ctx.corpus.lookup(pname) : null;
7              if (partner) {
8                  s.doc = {
9                      sees: [{
10                         children: [{
11                             kind: "reference",
12                             literal: pname,
13                             id: partner.id
14                         }]
15                     }]
16                 };
17             }
18         }
19     }
20 });
21
22 /**
23  * The name of a function's symmetric IO partner: `format_X` for a `parse_X`,
24  * and `parse_X` for a `format_X`. Returns null for any name that is neither
```

format_request

Format `r` as the wire-format text of an HTTP request.

Synopsis

Declared in `<parse-format-relates.cpp>`

```
char const*
format_request(request const& r);
```

Parameters

Name	Description
<code>r</code>	An HTTP request as a structured value.

See Also

[`parse\_request`](#)

format_user

Format `u` as the canonical wire-format text of a user.

Synopsis

Declared in `<parse-format-relates.cpp>`

```
char const*
format_user(user const& u);
```

Parameters

Name	Description
<code>u</code>	A user record.

See Also

[parse_user](#)

parse_request

Parse `text` into a request. Returns a valid request on success.

Synopsis

Declared in `<parse-format-relates.cpp>`

```
request  
parse_request(char const* text);
```

Return Value

An HTTP request as a structured value.

See Also

[`format\_request`](#)

parse_user

Parse `text` into a user.

Synopsis

Declared in `<parse-format-relates.cpp>`

```
user  
parse_user(char const* text);
```

Return Value

A user record.

See Also

[`format\_user`](#)

addons/extensions/ghostwriter.lua

```
1  local read_file, doc_path, paragraphs, blocks, inlines
2
3  mrdocs.register_transform("ghostwriter", function(ctx)
4      local root = (ctx.params and ctx.params.source) or "external-docs"
5      for _, sym in ipairs(ctx.corpus.symbols) do
6          -- Fill the description from the file when the symbol has none of its own.
7          -- The brief written in the header is a separate field, so it is kept; the
8          -- writer's file supplies the detailed description the developer left out.
9          if sym.name ~= nil and (not sym.doc or #(sym.doc.document or {}) == 0) then
10             local text = read_file(doc_path(ctx, sym, root))
11             if text then
12                 local new_blocks = blocks(paragraphs(text))
13                 if #new_blocks > 0 then
14                     if not sym.doc then
15                         sym.doc = {}
16                     end
17                     sym.doc.document = new_blocks
18                 end
19             end
20         end
21     end
22 end)
23
```

addons/extensions/ghostwriter.lua

```
1  local read_file, doc_path, paragraphs, blocks, inlines
2
3  mrdocs.register_transform("ghostwriter", function(ctx)
4      local root = (ctx.params and ctx.params.source) or "external-docs"
5      for _, sym in ipairs(ctx.corpus.symbols) do
6          -- Fill the description from the file when the symbol has none of its own.
7          -- The brief written in the header is a separate field, so it is kept; the
8          -- writer's file supplies the detailed description the developer left out.
9          if sym.name ~= nil and (not sym.doc or #(sym.doc.document or {}) == 0) then
10             local text = read_file(doc_path(ctx, sym, root))
11             if text then
12                 local new_blocks = blocks(paragraphs(text))
13                 if #new_blocks > 0 then
14                     if not sym.doc then
15                         sym.doc = {}
16                     end
17                     sym.doc.document = new_blocks
18                 end
19             end
20         end
21     end
22 end)
23
24 --- Read a whole file into a string
```

addons/extensions/ghostwriter.lua

```
5   for _, sym in ipairs(ctx.corpus.symbols) do
6       -- Fill the description from the file when the symbol has none of its own.
7       -- The brief written in the header is a separate field, so it is kept; the
8       -- writer's file supplies the detailed description the developer left out.
9       if sym.name ~= nil and (not sym.doc or #(sym.doc.document or {}) == 0) then
10          local text = read_file(doc_path(ctx, sym, root))
11          if text then
12              local new_blocks = blocks(paragraphs(text))
13              if #new_blocks > 0 then
14                  if not sym.doc then
15                      sym.doc = {}
16                  end
17                  sym.doc.document = new_blocks
18              end
19          end
20      end
21  end
22 end)
23
24 --- Read a whole file into a string.
25 --- @param path string Path to the file, relative to the run's working directory.
26 --- @return string|nil The file's contents, or nil when the file does not exist.
27 function read_file(path)
28     local f = io.open(path, "r")
29     if not f then
```

simple.cpp

```
1  /// Application value types.
2  namespace app {
3
4  /// A point or displacement in the plane.
5  struct Vec2
6  {
7      /// The x component.
8      double x;
9      /// The y component.
10     double y;
11
12     /// The Euclidean length.
13     ///
14     /// @return The straight-line length of the vector.
15     double length() const;
16 };
17
18 /// A circle in the plane.
19 struct Circle
20 {
21     /// The radius.
22     double radius;
23
24     /// The center point.
```

external-docs/app/Vec2.md

```
1 A `Vec2` stores a pair of Cartesian coordinates and doubles as both a point
2 and a displacement. Arithmetic on it follows the usual rules for a 2-D vector
3 space, so adding two vectors adds their components and scaling multiplies them.
4
5 Prefer `Vec2` over a bare `std::pair<double, double>` when the values are
6 geometric: the named `x` and `y` members and the geometry helpers make intent
7 obvious at the call site, and keep unrelated pairs from being mixed in by
8 mistake.
```

app::Vec2

A point or displacement in the plane.

Synopsis

Declared in `<simple.cpp>`

```
struct Vec2;
```

Description

A `Vec2` stores a pair of Cartesian coordinates and doubles as both a point and a displacement. Arithmetic on it follows the usual rules for a 2-D vector space, so adding two vectors adds their components and scaling multiplies them.

Prefer `Vec2` over a bare `std::pair<double, double>` when the values are geometric: the named `x` and `y` members and the geometry helpers make intent obvious at the call site, and keep unrelated pairs from being mixed in by mistake.

Member Functions

Name	Description
<code>length</code>	The Euclidean length.

Data Members

Name	Description
<code>x</code>	The x component.
<code>y</code>	The y component.

Non-Member Functions

Name	Description
------	-------------

Script Generators

simple.cpp

```
1  /// A vector in the Euclidean plane.
2  struct Vector
3  {
4      /** The length (magnitude) of the vector.
5
6          @return The Euclidean length.
7      */
8      double length() const;
9
10     /** Scale the vector componentwise.
11
12         @param sx Factor applied to the x component.
13         @param sy Factor applied to the y component.
14     */
15     void scale(double sx, double sy);
16 };
```

addons/extensions/search_index.lua

```
1  local url_for
2
3  mrdocs.register_generator("search-index", function(ctx)
4      local entries = {}
5      for _, sym in ipairs(ctx.corpus.symbols) do
6          local name = sym.name or ""
7          if name ~= "" then
8              local entry = {}
9              entry.name = name
10             entry.url = url_for(ctx.corpus, sym)
11             entries[#entries + 1] = entry
12         end
13     end
14     ctx.output.write("search-index.json", ctx.stringify(entries))
15 end)
16
17 --- A symbol's page URL: the chain of anchors from the global namespace down to
18 --- the symbol, joined by "/", plus ".html". Walk up through `parent` (a symbol
19 --- id) with `corpus.get`; the global namespace has no parent, so it contributes
20 --- no path segment.
21 --- @param corpus table `ctx.corpus`, used to resolve each `parent` id.
22 --- @param sym table The symbol to build a URL for.
23 --- @return string url The symbol's page URL.
```

addons/extensions/search_index.lua

```
2
3  mrdocs.register_generator("search-index", function(ctx)
4      local entries = {}
5      for _, sym in ipairs(ctx.corpus.symbols) do
6          local name = sym.name or ""
7          if name ~= "" then
8              local entry = {}
9              entry.name = name
10             entry.url = url_for(ctx.corpus, sym)
11             entries[#entries + 1] = entry
12         end
13     end
14     ctx.output.write("search-index.json", ctx.stringify(entries))
15 end)
16
17 --- A symbol's page URL: the chain of anchors from the global namespace down to
18 --- the symbol, joined by "/", plus ".html". Walk up through `parent` (a symbol
19 --- id) with `corpus.get`; the global namespace has no parent, so it contributes
20 --- no path segment.
21 --- @param corpus table `ctx.corpus`, used to resolve each `parent` id.
22 --- @param sym table The symbol to build a URL for.
23 --- @return string url The symbol's page URL.
24 function url_for(corpus, sym)
25     local parts = {}
26     local cur = sym
```

search-index.json

```
1  [  
2    {  
3      "name": "Vector",  
4      "url": "Vector.html"  
5    },  
6    {  
7      "name": "length",  
8      "url": "Vector/length.html"  
9    },  
10   {  
11     "name": "scale",  
12     "url": "Vector/scale.html"  
13   }  
14 ]
```

addons/extensions/toc.js

```
1  mrdocs.register_generator("toc", function(ctx) {
2      const roots = buildRoots(ctx.corpus);
3      ctx.output.write("toc.md", markdown(roots, 0));
4      ctx.output.write("toc.adoc", asciidoc(roots, 0));
5      ctx.output.write("toc.html", html(roots));
6  });
7
8  /**
9   * Build the top-level navigation nodes for the whole corpus: the page-level
10   * children of the global namespace (the one symbol with no `parent`), each
11   * expanded into a node tree.
12   * @param {object} corpus - `ctx.corpus`.
13   * @returns {object[]} The root nav nodes.
14   */
15  function buildRoots(corpus) {
16      let root = null;
17      for (let i = 0; i < corpus.symbols.length; ++i) {
18          if (!corpus.symbols[i].parent) {
19              root = corpus.symbols[i];
20              break;
21          }
22      }
23      const roots = [];
```

addons/extensions/toc.js

```
127     out += asciidoc(n.children, depth + 1);
128     out += asciidoc(n.children, depth + 1);
129 }
130 return out;
131 }
132
133 /**
134  * Render nav nodes as a nested HTML `<ul>` list. Returns an empty string for
135  * an empty list, so leaf nodes emit no child `<ul>`.
136  * @param {object[]} nodes - Nav nodes.
137  * @returns {string} The HTML list.
138  */
139 function html(nodes) {
140     if (nodes.length === 0) {
141         return "";
142     }
143     let out = "<ul>\n";
144     for (let i = 0; i < nodes.length; ++i) {
145         const n = nodes[i];
146         out += "<li><a href=\"\" + n.url + \"\">\" + n.name + "</a>\" + html(n.children) + "</li>\n";
147     }
148     out += "</ul>\n";
149     return out;
150 }
```

addons/extensions/toc.js

```
128     out += asciidoc(n.children, depth + 1);
129 }
130 return out;
131 }
132
133 /**
134  * Render nav nodes as a nested HTML `

` list. Returns an empty string for
135  * an empty list, so leaf nodes emit no child `

`'.
136  * @param {object[]} nodes - Nav nodes.
137  * @returns {string} The HTML list.
138  */
139 function html(nodes) {
140     if (nodes.length === 0) {
141         return "";
142     }
143     let out = "<ul>\n";
144     for (let i = 0; i < nodes.length; ++i) {
145         const n = nodes[i];
146         out += "<li><a href=\"\" + n.url + \"\">\" + n.name + "</a>\" + html(n.children) + "</li>\n";
147     }
148     out += "</ul>\n";
149     return out;
150 }
```

addons/extensions/toc.js

```
128     out += asciidoc(n.children, depth + 1);
129 }
130 return out;
131 }
132
133 /**
134  * Render nav nodes as a nested HTML `

` list. Returns an empty string for
135  * an empty list, so leaf nodes emit no child `

`.
136  * @param {object[]} nodes - Nav nodes.
137  * @returns {string} The HTML list.
138  */
139 function html(nodes) {
140     if (nodes.length === 0) {
141         return "";
142     }
143     let out = "<ul>\n";
144     for (let i = 0; i < nodes.length; ++i) {
145         const n = nodes[i];
146         out += "<li><a href=\"\" + n.url + \"\">\" + n.name + "</a>\" + html(n.children) + "</li>\n";
147     }
148     out += "</ul>\n";
149     return out;
150 }
```

toc.md

- 1 - [geo](geo.html)
- 2 - [coord](geo/coord.html)
- 3 - [from_polar](geo/coord/from_polar.html)
- 4 - [Point](geo/Point.html)
- 5 - [Relation](geo/Relation.html)
- 6 - [midpoint](geo/midpoint.html)

toc.adoc

```
1 * xref:geo.html[geo]
2 ** xref:geo/coord.html[coord]
3 *** xref:geo/coord/from_polar.html[from_polar]
4 ** xref:geo/Point.html[Point]
5 ** xref:geo/Relation.html[Relation]
6 ** xref:geo/midpoint.html[midpoint]
```

toc.html

```
1  <ul>
2  <li><a href="geo.html">geo</a><ul>
3  <li><a href="geo/coord.html">coord</a><ul>
4  <li><a href="geo/coord/from_polar.html">from_polar</a></li>
5  </ul>
6  </li>
7  <li><a href="geo/Point.html">Point</a></li>
8  <li><a href="geo/Relation.html">Relation</a></li>
9  <li><a href="geo/midpoint.html">midpoint</a></li>
10 </ul>
11 </li>
12 </ul>
```

addons/extensions/describe.js

```
1  mrdocs.register_generator("describe", function(ctx) {
2    const grouped = groupByHeader(ctx.corpus);
3    for (let i = 0; i < grouped.order.length; ++i) {
4      const header = grouped.order[i];
5      const group = grouped.byHeader[header];
6      ctx.output.write(sidecarName(header), renderSidecar(header, group.includes, group.order, group.byNs));
7    }
8  });
9
10 /**
11  * Group the describe annotation of every record and enum in the corpus, first
12  * by the header it is defined in and then by its enclosing namespace, keeping
13  * first-seen order at every level. Each header group also collects the base
14  * sidecars its types depend on, minus its own.
15  * @param {object} corpus - `ctx.corpus`.
16  * @returns {{order: string[], byHeader: Object<string, {order: string[], byNs: Object<string, string[]>, includes:
17  *   string[]}>}}}
18  *   The headers in first-seen order and, for each, its namespaces, their
19  *   annotation lines, and the base sidecars to include.
20  */
21 function groupByHeader(corpus) {
22   const order = [];
23   const byHeader = {};
24   for (let i = 0; i < corpus.headers.length; ++i) {
25     const header = corpus.headers[i];
26     const group = byHeader[header] || {order: [], byNs: {}, includes: []};
27     group.order.push(header);
28     for (let j = 0; j < header.namespaces.length; ++j) {
29       const ns = header.namespaces[j];
30       const nsGroup = group.byNs[ns] || {order: [], includes: []};
31       nsGroup.order.push(ns);
32       group.byNs[ns] = nsGroup;
33     }
34     group.includes.push(...header.sidecars);
35     for (let k = 0; k < header.types.length; ++k) {
36       const type = header.types[k];
37       const typeGroup = group.byHeader[type.header] || {order: [], byNs: {}, includes: []};
38       typeGroup.order.push(type.header);
39       for (let l = 0; l < type.namespaces.length; ++l) {
40         const ns = type.namespaces[l];
41         const nsGroup = typeGroup.byNs[ns] || {order: [], includes: []};
42         nsGroup.order.push(ns);
43         typeGroup.byNs[ns] = nsGroup;
44       }
45       typeGroup.includes.push(...type.sidecars);
46     }
47   }
48   return {order, byHeader};
49 }
```


addons/extensions/describe.js

```
191  */
192  function renderSidecar(header, includes, nsOrder, byNs) {
193      const guard = guardName(sidecarName(header));
194      let out =
195          "#ifndef " + guard + "\n" +
196          "#define " + guard + "\n\n" +
197          "// Generated by MrDocs from documented types. Do not edit by hand.\n" +
198          "//\n" +
199          "// Include this to gain Boost.Describe reflection over the types declared\n" +
200          "// in " + header + ".\n\n" +
201          "#include \"" + header + "\"\n";
202      for (let i = 0; i < includes.length; ++i) {
203          out += "#include \"" + includes[i] + "\"\n";
204      }
205      out += "#include <boost/describe.hpp>\n\n";
206      for (let g = 0; g < nsOrder.length; ++g) {
207          const ns = nsOrder[g];
208          if (ns) {
209              out += "namespace " + ns + " {\n\n";
210          }
211          out += byNs[ns].join("\n") + "\n";
212          if (ns) {
213              out += "\n} // namespace " + ns + "\n";
214          }
215          out += "\n".
```

addons/extensions/describe.js

```
14  * sidecars its types depend on, minus its own.
15  * @param {object} corpus - `ctx.corpus`.
16  * @returns {{order: string[], byHeader: Object<string, {order: string[], byNs: Object<string, string[]>, includes:
    string[]}>}}
17  *   The headers in first-seen order and, for each, its namespaces, their
18  *   annotation lines, and the base sidecars to include.
19  */
20  function groupByHeader(corpus) {
21    const order = [];
22    const byHeader = {};
23    for (let i = 0; i < corpus.symbols.length; ++i) {
24      const sym = corpus.symbols[i];
25      const line = describeMacro(corpus, sym);
26      if (line === null) {
27        continue;
28      }
29      const header = sourceHeaderOf(sym);
30      if (!header) {
31        continue;
32      }
33      if (!(header in byHeader)) {
34        byHeader[header] = { order: [], byNs: {}, includes: [] };
35        order.push(header);
36      }
37      const group = byHeader[header].
```

circle.hpp

```
1  #ifndef APP_CIRCLE_HPP
2  #define APP_CIRCLE_HPP
3
4  #include "shape.hpp"
5
6  namespace app {
7
8  /// A circle.
9  struct Circle : Shape
10 {
11     /// The radius.
12     double radius;
13
14     /// The enclosed area.
15     double area() const;
16 };
17
18 } // namespace app
19
20 #endif
```

Generated sidecars

```
1  #ifndef CIRCLE_DESCRIBED_HPP
2  #define CIRCLE_DESCRIBED_HPP
3
4  // Generated by MrDocs from documented types. Do not edit by hand.
5  //
6  // Include this to gain Boost.Describe reflection over the types declared
7  // in circle.hpp.
8
9  #include "circle.hpp"
10 #include "shape.described.hpp"
11 #include <boost/describe.hpp>
12
13 namespace app {
14
15 BOOST_DESCRIBE_STRUCT(Circle, (Shape), (radius, area))
16
17 } // namespace app
18
19 #endif // CIRCLE_DESCRIBED_HPP
```

```
addons/extensions/snippets.lua
```

addons/extensions/snippets.lua

```
8  -- the generator reads it.
9  local imported = {}
10
11  -- Import: for each symbol with no example of its own, append the snippets tagged
12  -- with its name in `<source>/<header-stem>.cpp` to its documentation, after any
13  -- description it already carries, and remember them so export skips them.
14  mrdocs.register_transform("snippets", function(ctx)
15      local root = (ctx.params and ctx.params.source) or "example-code"
16      local files = {}
17      for _, sym in ipairs(ctx.corpus.symbols) do
18          if sym.name ~= nil and #code_blocks(sym) == 0 then
19              local header = header_of(sym)
20              if header then
21                  local path = root .. "/" .. stem_of(header) .. ".cpp"
22                  if files[path] == nil then
23                      files[path] = read_file(path) or false
24                  end
25                  local text = files[path]
26                  if text then
27                      local snippets = extract_snippets(text, sym.name)
28                      if #snippets > 0 then
29                          if not sym.doc then
30                              sym.doc = {}
31                          end
32                          -- A script assigns a fresh block list rather than mutating one in
```

addons/extensions/snippets.lua

```
15     local root = (ctx.params and ctx.params.source) or "example-code"
16     local files = {}
17     for _, sym in ipairs(ctx.corpus.symbols) do
18         if sym.name ~= nil and #code_blocks(sym) == 0 then
19             local header = header_of(sym)
20             if header then
21                 local path = root .. "/" .. stem_of(header) .. ".cpp"
22                 if files[path] == nil then
23                     files[path] = read_file(path) or false
24                 end
25                 local text = files[path]
26                 if text then
27                     local snippets = extract_snippets(text, sym.name)
28                     if #snippets > 0 then
29                         if not sym.doc then
30                             sym.doc = {}
31                         end
32                         -- A script assigns a fresh block list rather than mutating one in
33                         -- place, so rebuild the description and append the imported snippets.
34                         local doc = {}
35                         for _, b in ipairs(sym.doc.document or {}) do
36                             doc[#doc + 1] = to_plain(b)
37                         end
38                         -- MrDocs heads an inline @code example with an "Example"/"Examples"
39                         -- heading: add the same before the imported snippets so they read
```

addons/extensions/snippets.lua

```
35     for _, b in ipairs(sym.doc.document or {}) do
36         doc[#doc + 1] = to_plain(b)
37     end
38     -- MrDocs heads an inline @code example with an "Example"/"Examples"
39     -- heading; add the same before the imported snippets so they read
40     -- the same as inline ones.
41     local label = (#snippets == 1) and "Example" or "Examples"
42     doc[#doc + 1] = {
43         kind = "heading",
44         children = { { kind = "text", literal = label } },
45     }
46     for _, snippet in ipairs(snippets) do
47         doc[#doc + 1] = { kind = "code", literal = snippet }
48         mark_imported(snippet, header)
49     end
50     sym.doc.document = doc
51 end
52 end
53 end
54 end
55 end
56 end)
57
58 -- Export: group every symbol's inline @code examples by header and write one
59 -- runnable test file per header under `<output>/<header-stem>.cpp` one function
```

addons/extensions/snippets.lua

```
49         end
50         sym.doc.document = doc
51     end
52 end
53 end
54 end
55 end
56 end)
57
58 -- Export: group every symbol's inline @code examples by header and write one
59 -- runnable test file per header under `<output>/<header-stem>.cpp`, one function
60 -- per symbol plus a `main` that calls them all.
61 mrdocs.register_generator("snippets", function(ctx)
62     local out_dir = (ctx.params and ctx.params.output) or "exported"
63     local groups = {}
64     local order = {}
65     for _, sym in ipairs(ctx.corpus.symbols) do
66         if sym.name ~= nil then
67             local header = header_of(sym)
68             if header then
69                 -- Export only the snippets written inline in the header; the ones the
70                 -- transform pulled from example files are already tested where they live.
71                 local blocks = {}
72                 for _, literal in ipairs(code_blocks(sym)) do
73                     if not was_imported(literal, header) then
```

addons/extensions/snippets.lua

```
62     local out_dir = (ctx.params and ctx.params.output) or "exported"
63     local groups = {}
64     local order = {}
65     for _, sym in ipairs(ctx.corpus.symbols) do
66         if sym.name ~= nil then
67             local header = header_of(sym)
68             if header then
69                 -- Export only the snippets written inline in the header; the ones the
70                 -- transform pulled from example files are already tested where they live.
71                 local blocks = {}
72                 for _, literal in ipairs(code_blocks(sym)) do
73                     if not was_imported(literal, header) then
74                         blocks[#blocks + 1] = literal
75                     end
76                 end
77                 if #blocks > 0 then
78                     if not groups[header] then
79                         groups[header] = {}
80                         order[#order + 1] = header
81                     end
82                     local g = groups[header]
83                     g[#g + 1] = { name = sym.name, line = line_of(sym), literals = blocks }
84                 end
85             end
86         end
87     end
```

addons/extensions/snippets.lua

```
69      -- Export only the snippets written inline in the header; the ones the
70      -- transform pulled from example files are already tested where they live.
71      local blocks = {}
72      for _, literal in ipairs(code_blocks(sym)) do
73          if not was_imported(literal, header) then
74              blocks[#blocks + 1] = literal
75          end
76      end
77      if #blocks > 0 then
78          if not groups[header] then
79              groups[header] = {}
80              order[#order + 1] = header
81          end
82          local g = groups[header]
83          g[#g + 1] = { name = sym.name, line = line_of(sym), literals = blocks }
84      end
85      end
86      end
87  end
88  for _, header in ipairs(order) do
89      local syms = groups[header]
90      table.sort(syms, function(a, b) return a.line < b.line end)
91      local seen = {}
92      local parts = { "#include \"" .. header .. "\"\n#include <cassert>\n" }
```

addons/extensions/snippets.lua

```
83         g[#g + 1] = { name = sym.name, line = line_of(sym), literals = blocks }
84     end
85 end
86 end
87 end
88 for _, header in ipairs(order) do
89     local syms = groups[header]
90     table.sort(syms, function(a, b) return a.line < b.line end)
91     local seen = {}
92     local parts = { "#include \"" .. header .. "\"\n#include <cassert>\n" }
93     local calls = {}
94     for _, sym in ipairs(syms) do
95         local fn, def = symbol_function(sym.name, sym.literals, seen)
96         parts[#parts + 1] = def
97         calls[#calls + 1] = "    " .. fn .. "();"
98     end
99     parts[#parts + 1] = "int main()\n{\n" .. table.concat(calls, "\n") .. "\n}\n"
100    ctx.output.write(out_dir .. "/" .. stem_of(header) .. ".cpp", table.concat(parts, "\n"))
101 end
102 end)
103
104 --- The code-block literals in a symbol's documentation, in order. A `@code`
105 --- example, whether written inline or imported, is a `code` block in the
106 --- symbol's `document`.
```

arithmetic.hpp

```
1  #ifndef APP_ARITHMETIC_HPP
2  #define APP_ARITHMETIC_HPP
3
4  namespace app {
5
6  /** Add two integers.
7
8      Returns the arithmetic sum of the two arguments. The example lives in this
9      comment and is exported to a file the build compiles, so it never falls out
10     of date.
11
12     @par Example
13
14     @code
15     int s = app::add(2, 3);
16     assert(s == 5);
17     @endcode
18  */
19  int add(int a, int b);
20
21  /** Multiply two integers.
22
23      Returns the product of the two arguments. Its examples are maintained in a
```

arithmetic.hpp

```
14     @code
15     int s = app::add(2, 3);
16     assert(s == 5);
17     @endcode
18 */
19 int add(int a, int b);
20
21 /** Multiply two integers.
22 
23     Returns the product of the two arguments. Its examples are maintained in a
24 separate file.
25 */
26 int multiply(int a, int b);
27
28 /** Subtract the second integer from the first.
29 
30     Returns `a - b`. Its example is maintained in a separate file.
31 */
32 int subtract(int a, int b);
33
34 } // namespace app
35
36 #endif
```

example-code/arithmetic.cpp

```
1  #include "arithmetic.hpp"
2  #include <cassert>
3
4  void examples()
5  {
6      //[multiply
7      int p = app::multiply(4, 5);
8      assert(p == 20);
9      //]
10
11     //[multiply
12     int q = app::multiply(-2, 3);
13     assert(q == -6);
14     //]
15
16     //[subtract
17     int d = app::subtract(9, 4);
18     assert(d == 5);
19     //]
20 }
```

app::add

Add two integers.

Synopsis

Declared in `<arithmetic.hpp>`

```
int
add(
    int a,
    int b);
```

Description

Returns the arithmetic sum of the two arguments. The example lives in this comment and is exported to a file the build compiles, so it never falls out of date.

Example

```
int s = app::add(2, 3);
assert(s == 5);
```

app::multiply

Multiply two integers.

Synopsis

Declared in `<arithmetic.hpp>`

```
int
multiply(
    int a,
    int b);
```

Description

Returns the product of the two arguments. Its examples are maintained in a separate file.

Examples

```
int p = app::multiply(4, 5);
assert(p == 20);
```

```
int q = app::multiply(-2, 3);
assert(q == -6);
```

Exported test files

```
1  #include "arithmetic.hpp"
2  #include <cassert>
3
4  void add_snippet()
5  {
6      int s = app::add(2, 3);
7      assert(s == 5);
8  }
9
10 int main()
11 {
12     add_snippet();
13 }
```

Mr.Docs as a Library

examples/library/breaking-changes/CMakeLists.txt

```
1 find_package(mrdocs REQUIRED CONFIG)
2
3 add_executable(mrdocs-breaking-changes-example
4     src/main.cpp
5     src/Corpus.cpp
6     src/Diff.cpp
7     src/BreakingChangesGenerator.cpp
8 )
9 target_link_libraries(mrdocs-breaking-changes-example PRIVATE mrdocs::mrdocs-core)
```

examples/library/breaking-changes/src/Corpus.cpp

```
1 Expected<Corpus>
2 loadCorpusFromConfig(
3     std::string const& configPath,
4     ReferenceDirectories const& dirs,
5     char const** argv)
6 {
7     Config config;
8     ReferenceDirectories localDirs = dirs;
9     localDirs.cwd = std::string(files::getParentDir(configPath));
10    // Command-line overrides (if any) are applied on top of the config file and
11    // before normalization. Running the example against an installed MrDocs
12    // needs none; the in-tree test forwards the built-in directory flags here.
13    MRDOCS_TRY(Config::load_file(config, configPath, localDirs, argv));
14    // load_file applies the config's log level; this example only wants the
15    // breaking-change report, so keep extraction diagnostics quiet.
16    report::setMinimumLevel(report::Level::error);
17    return Corpus::build(config);
18 }
```

examples/library/breaking-changes/src/Diff.cpp

```
1  DiffResult
2  diff(Corpus const& v1, Corpus const& v2)
3  {
4      DiffResult result;
5
6      // For every public symbol in v1, see what happened in v2.
7      for (Symbol const& a : v1)
8      {
9          if (!isPublicTopLevel(a))
10         {
11             continue;
12         }
13         std::string name = v1.qualifiedName(a);
14         auto found = v2.lookup(name);
15         if (!found)
16         {
17             result.removed.emplace_back(std::move(name), &a);
18             continue;
19         }
20         Symbol const& b = *found;
21
22         if (a.Kind != b.Kind)
23         {
```

examples/library/breaking-changes/src/Diff.cpp

```
19         }
20         Symbol const& b = *found;
21
22         if (a.Kind != b.Kind)
23         {
24             // A function turned into a record, or similar.
25             // Treat it as removed + added rather than changed.
26             result.added.emplace_back(name, &b);
27             result.removed.emplace_back(std::move(name), &a);
28             continue;
29         }
30
31         if (a.isFunction())
32         {
33             auto reasons = compareFunctions(a.asFunction(), b.asFunction());
34             if (!reasons.empty())
35             {
36                 result.changed.push_back(
37                     {std::move(name), &a, &b, std::move(reasons)});
38             }
39         }
40     }
41
42     // Anything public in v2 that wasn't in v1 is a new addition.
```

examples/library/breaking-changes/src/Diff.cpp

```
37             {std::move(name), &a, &b, std::move(reasons)}});
38         }
39     }
40 }
41
42     // Anything public in v2 that wasn't in v1 is a new addition.
43     for (Symbol const& b : v2)
44     {
45         if (!isPublicTopLevel(b))
46         {
47             continue;
48         }
49         std::string name = v2.qualifiedName(b);
50         if (!v1.lookup(name))
51         {
52             result.added.emplace_back(std::move(name), &b);
53         }
54     }
55
56     if (!result.removed.empty() || !result.changed.empty())
57     {
58         result.impact = SemverImpact::Major;
59     }
60     else if (!result.added.empty())
```

examples/library/breaking-changes/src/Diff.cpp

```
43     for (Symbol const& b : v2)
44     {
45         if (!isPublicTopLevel(b))
46         {
47             continue;
48         }
49         std::string name = v2.qualifiedName(b);
50         if (!v1.lookup(name))
51         {
52             result.added.emplace_back(std::move(name), &b);
53         }
54     }
55
56     if (!result.removed.empty() || !result.changed.empty())
57     {
58         result.impact = SemverImpact::Major;
59     }
60     else if (!result.added.empty())
61     {
62         result.impact = SemverImpact::Minor;
63     }
64     return result;
65 }
```

examples/library/breaking-changes/src/BreakingChangesGenerator.hpp

```
1  class BreakingChangesGenerator final : public Generator
2  {
3  public:
4      explicit BreakingChangesGenerator(Corpus const& baseline)
5          : baseline_(&baseline)
6      {}
7
8      std::string_view id()          const noexcept override;
9      std::string_view displayName() const noexcept override;
10     std::string_view fileExtension()const noexcept override;
11
12     Expected<void>
13     build(Corpus const& current, Config const& config) const override;
14
15 private:
16     Corpus const* baseline_;
17 };
```

examples/library/breaking-changes/src/BreakingChangesGenerator.cpp

```
1  // build() does whatever the generator needs. This one is not a  
2  // file-producing documentation format: it diffs the baseline against the  
3  // candidate corpus and prints the report to stdout. A generator is free  
4  // to bring its output to the screen instead of writing files.  
5  Expected<void>  
6  BreakingChangesGenerator::  
7  build(Corpus const& current, Config const& config) const  
8  {  
9      writeReport(std::cout, diff(*baseline_, current));  
10     return {};  
11 }
```

examples/library/breaking-changes/test/v1/lib.cpp

```
1  /// A point on the plane.
2  struct Point
3  {
4      /// X coordinate.
5      int x;
6
7      /// Y coordinate.
8      int y;
9
10     /** Distance from the origin.
11
12         @return The Euclidean length of the vector.
13     */
14     double length() const;
15 };
16
17 /// Compute the area of a circle.
18 double area_of_circle(double radius);
19
20 /// Helper that should be removed in v2.
21 int legacy_helper();
```

examples/library/breaking-changes/test/v2/lib.cpp

```
1  /// A point on the plane.
2  struct Point
3  {
4      /// X coordinate.
5      int x;
6
7      /// Y coordinate.
8      int y;
9
10     /** Distance from the origin.
11
12         @return The Euclidean length of the vector.
13     */
14     double length() const noexcept;
15 };
16
17 /// Compute the area of a circle.
18 double area_of_circle(double radius);
19
20 /// New helper added in v2.
21 double area_of_square(double side);
```

Terminal

```
1  == Breaking-change report ==
2
3  Added (1):
4    + [function] area_of_square
5
6  Removed (1):
7    - [function] legacy_helper
8
9  Changed (1):
10   * [function] Point::length
11     - noexcept differs
12
13 Suggested version bump: major
```



Boost.URL

Quick Look

URLs

- Parsing
- Components
- Segments
- Params

Operations

- Normalization
- String Token
- Percent Encoding
- Formatting

Customization

- Parse Rules
- Character Sets
- Compound Rules
- Ranges
- RFC 3986

Examples

- QR Code
- Finicky
- mailto URLs
- Magnet Link
- File Router
- Router
- Sanitizing URLs

Design Rationale

Reference

Help Card



boost::urls::url_view



boost::urls::url_view

A non-owning reference to a valid URL

Synopsis

Declared in `<boost/url/url_view.hpp>`

```
class url_view
    : public url_view_base
```

Description

Objects of this type represent valid URL strings constructed from a parsed, external character buffer whose storage is managed by the caller. That is, it acts like a `core::string_view` in terms of ownership. The caller is responsible for ensuring that the lifetime of the underlying character buffer extends until it is no longer referenced.

Example 1

Construction from a string parses the input as a *URI-reference* and throws an exception on error. Upon success, the constructed object points to the passed character buffer; ownership is not transferred.

```
url_view u( "https://www.example.com/index.htm?text=none#a1" );
```

Example 2

Parsing functions like `parse_uri_reference` return a `boost::system::result` containing either a valid `url_view` upon success, otherwise they contain an error. The error can be converted to an exception by the caller if desired:

```
system::result< url_view > rv = parse_uri_reference( "https://www.example.com/index.htm?text=none#a1" );
```

BNF

	Mr.Docs	Other	By hand	None
Convenient (author)	✓	?	×	✓
Friendly output (user)	✓	?	✓	×
AI-friendly	✓	✓	×	×
IDE integration	✓	✓	×	×
Verifiable	✓	×	×	×
In sync with source	✓	?	×	×
Single source of truth	✓	×	×	×
Extensions	✓	×	×	×



mrdocs.com



MR.DOCS: NATIVE C++. NO WORKAROUNDS.

MrDocs: Reliable Reference Documentation for C++ Using the Clang AST

ALAN DE FREITAS

